



Data formats in analytical DBMSs: performance trade-offs and future directions

Chunwei Liu¹ · Anna Pavlenko² · Matteo Interlandi² · Brandon Haynes²

Received: 20 September 2024 / Revised: 20 February 2025 / Accepted: 25 February 2025 / Published online: 19 March 2025
© The Author(s) 2025

Abstract

This paper evaluates the suitability of Apache Arrow, Parquet, and ORC as formats for subsumption in an analytical DBMS. We systematically identify and explore the high-level features that are important to support efficient querying in modern OLAP DBMSs and evaluate the ability of each format to support these features. We find that each format has trade-offs that make it more or less suitable for use as a format in a DBMS and identify opportunities to more holistically co-design a unified in-memory and on-disk data representation. Notably, for certain popular machine learning tasks, none of these formats perform optimally, highlighting significant opportunities for advancing format design. Our hope is that this study can be used as a guide for system developers designing and using these formats, as well as provide the community with directions to pursue for improving these common open formats.

1 Introduction

Over the last decade, a number of new common and open formats have been proposed that purport to improve performance and ease interoperability across OLAP DBMSs and storage layers such as data lakes [1]. Today, storage formats such as Parquet [2] and ORC [3] are the cornerstone reference architectures for cloud-scale data warehousing systems [4]. At the same time, the in-memory format Apache Arrow [5] is widely considered to be the default means of interoperability across different data systems [6], and several systems are even exploring how to leverage it end-to-end [7]. Each of these in-memory and storage formats attempt to minimize disk, memory, and IO costs, and each applies a wide variety of optimizations to maximize analytic read performance.

Though the benefits for common open formats are now well established [8], there has been less exploration of the

relative benefits of these formats for *direct subsumption* in an analytical DBMS as a native format. This is despite the robust discussion in database community about the relative merits of these formats for this purpose (e.g., [9–11]). One reason for this is that each format makes design choices that optimize for its use as a common and open format, and these choices often conflict with longstanding analytical DBMS techniques. For example, we increasingly see a push to directly leverage Arrow as an end-to-end, in-memory format within a DBMS [12]. At the same time, DBMS in-memory columnar formats typically encode data [13, 14] to minimize space and reduce memory requirements. However, Apache Arrow by default provides no encoding support, leaving it at odds with typical DBMS design. On the other hand, on-disk formats such as Parquet arrange data in a form that is much closer to that found in modern columnar DBMSs (e.g., by employing run-length encoding mixed with dictionary encoding and bit packing). However, Parquet is widely leveraged only as storage format and exposes no dedicated in-memory representation. Instead, developers bring Parquet data into memory and convert it to the Arrow format, which, as stated above, is suboptimal for a columnar DBMS. Finally, a format such as ORC at first glance appears to offer the best of both worlds, since it provides both an efficiently encoded data format and a related in-memory representation. Nevertheless, Arrow and Parquet are considered the standard nowadays because of their popularity in terms of activity in

✉ Chunwei Liu
chunwei@csail.mit.edu

Anna Pavlenko
annapa@microsoft.com

Matteo Interlandi
mainterl@microsoft.com

Brandon Haynes
brhaynes@microsoft.com

¹ MIT CSAIL, Cambridge, USA

² Microsoft, Redmond, USA

open-source projects and support from big data frameworks and large-scale query providers [15].

Given this environment, the goal of this paper is to evaluate these three formats, explore their trade-offs, and evaluate their performance as candidates for direct subsumption in an analytical DBMS. Three main challenges exist in subsuming an in-memory format such as Arrow or traditionally on-disk formats such as Parquet or ORC. First, a DBMS needs to be able to efficiently (de)serialize and (de)compress on-disk data to and from an in-memory representation. For this, efficiency directly depends on format's compression ratio, decompression speed, and transcoding performance. These trade-offs can be subtle and the line between an "in-memory" or "on-disk" format is often blurry. For example, in some cases a DBMS could improve performance by writing Arrow to disk or directly operating on Parquet in memory, avoiding transcoding costs and taking advantage of the features offered by each format. Additionally, for specialized machine learning tasks, such as retrieval augmented generation (RAG) and k -nearest neighbor search, current formats require significant improvements across all dimensions to support these tasks effectively.

Second, prior work has established that it is highly advantageous for a DBMS to "push down" computation as far as possible (e.g., to disk coprocessors or into the compressed domain) and to do so over as many data types as possible [16, 17]. Computation pushdown subsumes a number of related techniques. *Column pruning* and *data skipping* respectively enable a DBMS to avoid decompressing columns or rows that do not contain data relevant to a query answer (e.g., when executing a range query by skipping data regions that do not contain data within the range). Techniques such as *direct querying* enable a DBMS to retrieve query answers without an expensive decode or decompression step [18–20]. As we show in Table 1, the ability for common in-memory and on-disk formats to support these techniques is uneven; to maximize performance a DBMS should optimize for the resulting trade-offs.

Finally, to maximize performance, modern DBMSs leverage modern techniques such as vectorized execution (e.g., SIMD) [21–26] and query compilation [27]. Here again the ability for a format to support these techniques is uneven. For example, Parquet can vectorize operations over some query types whereas Arrow is inherently unencoded and more amenable to vectorized execution.

In summary, in this paper we present the first detailed, empirical evaluation of three popular and increasingly-adopted formats and evaluate their suitability to be used as a native format in a DBMS. Our hope is that this study can be used as a guide for system developers using these formats, as well as provide the community with directions to pursue for improving these common open formats.

Our contributions include:

- We succinctly summarize the design nuances and distinctions of three widely-adopted open columnar formats: Apache Arrow, Parquet and ORC (Sect. 3).
- We systematically identify and explore the high-level features that are important to support efficient querying in modern OLAP DBMSs (Sect. 4).
- For each format, we evaluate its ability to support efficient encoding, compression, and transcoding (Sect. 5) for both real-world, synthetic datasets, and various data types, including scalars and nested vectors.
- We benchmark the ability of each format to support select-project (SP) operations found near the leaves of query plans. We evaluate these in isolation (Sect. 6) and in combination (Sect. 7) using TPC-DS query plan fragments and over various data types.
- We evaluate the ability for each format to take advantage of recent trends such as vectorization, query compilation, and direct querying (Sect. 8).
- We demonstrate that each format's ability to operate on machine learning vector embeddings in terms of compression and (de)serialization is inadequate and suggest improvements to better-balance compression performance and task accuracy.
- We identify key opportunities to holistically co-design a unified in-memory and on-disk data representation.

2 Background: compression and data encoding

Data systems employ compression algorithms to reduce on-disk or in-memory data sizes and improve bandwidth utilization [28]. Conventional compression has traditionally focused on minimizing file size. This focus on size alone, while appropriate for storage, overlooks DBMS query execution performance [25, 29, 30]. Conversely, in an analytical columnar DBMS, compressed size is usually balanced with the ability to query directly on the compressed data.

2.1 Compression

Because of their generality, byte-oriented compression techniques (e.g., Gzip [31], Snappy [32], and Zlib [33]) are widely used to reduce data size [19, 30]. They treat the input values as a byte stream and compress them sequentially. Byte-oriented compression is applicable to all data types and, in general, exhibits good compression ratios [25]. However, these methods are computationally intensive [19]. A data block needs to be fully decompressed before individual values can be accessed. This often introduces unnecessary overhead for query execution.

Table 1 A comparison of the features found in common open columnar data formats

Feature	Arrow	Feather	Parquet	ORC
Encoding methods	DICT	DICT	DICT(-RLE), RLE, BP, Delta	DICT, RLE, BP, Delta
Compression codecs	None	Zstd, LZ4	Gzip, Snappy, Zstd, LZ4, (LZO)	Snappy, Zlib, LZ4
Skipping	Chunk-level	None	Record-level	Chunk-level
Direct query	None	None	None	None
Primary purpose	In-Memory Compute	On-Disk Storage	On-Disk Storage	On-Disk Storage
Representative systems	Dremio, Spark, Pandas	Pandas	Spark, Hive, Presto	Hive, Presto

2.2 Encoding

For decades, many data engines used row-oriented storage formats for OLTP query workloads [9]. As more complex OLAP workloads became common, *columnar storage formats* began to predominate [18]. Columnar databases store data of the same type together [34], allowing systems to leverage *lightweight compression*, also referred to as *encoding* [19]. Encoding methods such as *dictionary encoding*, *run-length encoding*, and *bit-packed encoding* are typically designed to compress a specific type of data, enabling efficient compression and better record-level access relative to the general-purpose compression approaches described in the previous section.

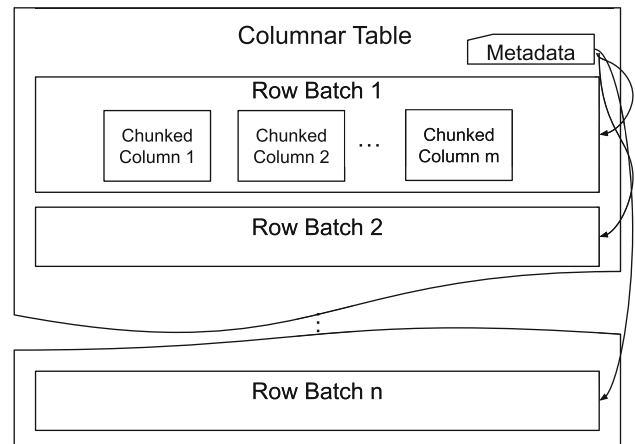
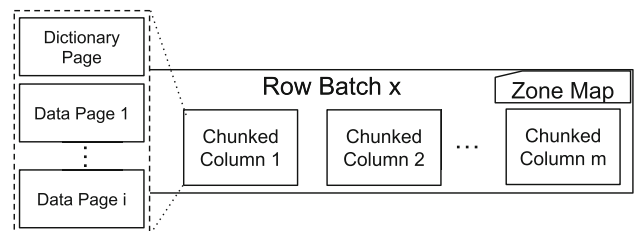
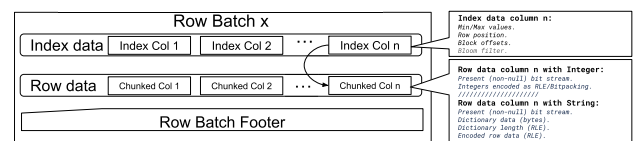
Some encoding methods also support direct querying and data skipping to improve query performance [19, 25, 35–37]. Systems such as Redshift [38] and SQL Server [39] support many lightweight compression approaches that reduce the storage cost of data; at the same time, they apply compression-specific optimizations to improve query execution performance. Previous research [26, 40–44] has also demonstrated that, for specific datasets, good encoding achieves a comparable compression ratio with far fewer CPU cycles than does byte-oriented compression algorithms.

We then provide an overview of several widely-used encoding algorithms discussed in this paper and highlight their applicable scenarios.

Bit-packed encoding (BP) works on numerical data. It finds the minimal number of bits needed to represent values and removes superfluous leading zeros. It works best when the target numbers have similar bit-width.

Dictionary encoding (DICT) works on all data types. It encodes each distinct entry with an integer key and bit-packs the integers. Dictionary encoding works best when the dataset has small cardinality and many repetitions. Queries on dictionary encoded data can be applied either on the fully decoded data or directly in the encoded domain after query rewriting using dictionary translation.

Run-length encoding (RLE) works on data with many consecutive repetitions. It replaces a run of the same value with a pair consisting of the value and how many times it is repeated.

**Fig. 1** Columnar format layout**Fig. 2** A Parquet row batch**Fig. 3** An ORC row batch

Hybrid encodings are derived from the above encoding techniques. Dictionary run-length encoding (DICT-RLE) applies RLE on the dictionary encoded keys to further compress data. Bit-packed and run-length hybrid encoding is used as a default implementation for the Parquet RLE encoder. Hybrid encoding usually achieves better compression performance at the cost of performance.

Table 2 Column format name convention mapping

	Row batch	Chunked column
Arrow	Record batch	Chunked array
Parquet	Row group	Column chunk
ORC	Stripe	Row column

3 Columnar open formats

In big data environments today, there are many optimized data formats for columnar data storage and computation, as we show in Table 1. Interestingly, these data formats share the same basic underlying design. Therefore, we begin by providing a “generic” architecture that summarizes the substantial commonality in modern columnar data format design (Sect. 3.1). Then, we drill into the idiosyncrasies found in Arrow, Parquet and ORC (Sects. 3.2, 3.3, and 3.4, respectively). To ease the parsing for non-familiar readers, we take the liberty of providing a unified naming convention. Table 2 has the mapping between our naming and each format.

3.1 Open columnar formats 101

Figure 1 summarizes a generic columnar format design. Columnar storage formats physically arrange data such that all the records belonging to the same column are stored sequentially. To achieve better data access at scale, columnar formats partition columns into *chunks*. *Chunked columns* are not created arbitrarily; instead, row-level alignment is attained by first splitting a table horizontally into *row batches* where, within each batch, rows are then partitioned into column chunks. Metadata about the row batches (e.g., their location, number, length, compression algorithm, etc.) are stored either into the footer or in the preamble of the file.

3.2 Apache arrow (Feather)

Apache Arrow [5] is a columnar data structure supporting efficient in-memory computing. Arrow can represent both flat and hierarchical data. Arrow is designed to be complementary to on-disk columnar data formats such as Parquet and ORC, and in fact it shares with them the same design depicted in Fig. 1. On-disk data files are decompressed, decoded, and loaded into Arrow in-memory columnar arrays. Each row batch has a default size of 64K rows. Arrow column chunks have a *present* bit-vector signaling whether a value is null (or not), and, for strings, optionally a dictionary.

The Arrow columnar format has some compelling properties: random access is $\mathcal{O}(1)$ for entries in the same chunked column, and each value cells are sequential in memory, so it's efficient to iterate over. Arrow also defines a binary seri-

alization protocol for converting a collection of row batches that can be used for messaging, interprocess communication (IPC), and writing blobs into storage. Deserializing an Arrow blob has effectively zero cost.

Closely related to the Arrow format, *Arrow Feather* [45] is a column-oriented binary disk-based format, leveraging the same IPC as the in-memory Arrow format. Additionally, Feather adds dictionary encoding (for strings) and compression (Zstd, LZ4). Datasets stored in Arrow Feather are loaded in-memory as *Arrow Tables*.

3.3 Parquet

Parquet [2] is a columnar-oriented storage format inspired by the nested data storage format outlined in Google Dremel [46]. Parquet integrates many efficient compression and encoding approaches to achieve space-efficiency. A Parquet file is structured almost exactly as described in Sect. 3.1; however, as illustrated in Fig. 2, each column chunk is partitioned into a *dictionary page* and series of *data pages*. Its file footer additionally contains *zone maps* (e.g., min, max, and number of NULLs) at the row batch, chunked column, and data page level. This enables efficient data skipping. Row batches have a recommended size of 512-1024 MB. Parquet applies dictionary encoding per data page and falls back to plain encoding when a dictionary grows larger than a predefined threshold. Parquet is designed to be space and IO-efficient at the expense of CPU utilization for decoding. It does not provide any data structures for in-memory computing.

3.4 ORC

Optimized Row Columnar (ORC) [3] is a storage format designed for read-heavy analytical workloads. ORC files are organized as in Fig. 1 where the default row batch size is 250 MB. Differently than Parquet, as is shown in Fig. 3, ORC organizes columns into an *index* that contains min/max values, bloom filters, etc., and *row data* with a *present* bit-vector indicating NULL entries. The chunked columns in the row data are formatted based on the encoding type.

ORC exposes a corresponding in-memory format, which contains a row-level index and NULL bit-vector data structures for fast querying and NULL checks. ORC supports dictionary encoding (at the row batch level) for string data. Similar to Parquet, ORC falls back to plain encoding when the number of distinct values is greater than a threshold (e.g., for Hive, 80% of the records).

3.5 Discussion

Overall, Parquet and ORC provide the most comprehensive compression support for common data types, whereas Arrow Feather supports the fewest. ORC provides more aux-

Table 3 Default encoding by format and data type

	Parquet	Arrow	ORC
Integer	DICT(-RLE)*	None	RLE
Double	DICT(-RLE)*	None	None
String/binary	DICT(-RLE)*	DICT	DICT-RLE
Decimal	DICT(-RLE)*	None	Int-RLE [†]
Vector (nested)	Nested encoding [◇]	Value vectors [★]	List [‡]

* Parquet uses DICT in the latest C++ API, while DICT-RLE was used in its legacy Java API. [†] Int-RLE refers to the encoding where the decimal value is scaled to an integer and then encoded with RLE encoding.

◇ Parquet uses the Dremel encoding with definition and repetition levels to encode nested vector columns. ★ Arrow's nested-data encoding (originated as a fork of Apache Drill's ValueVectors [47]) uses bitmaps for null tracking and offset buffers for variable-length elements, optimizing memory efficiency and SIMD support. ‡ ORC encodes a vector using its List type [3], which includes a presence stream, a length stream with RLE, and an element value stream that encodes based on the underlying type

iliary information for query execution (e.g., its zone map and support for bloom filters). Arrow Feather applies the same compression type to all arrays in the same record batch, whereas Parquet is more granular and allows compression to vary across column chunks. This flexibility enables intelligent encoding and compression selection based on the data features or workload characteristics [25]. As summarized in Table 3, each format applies different default encoding strategies.

In terms of data access, both Arrow and ORC require data to be fully loaded into dedicated in-memory data structures (an Arrow Table or ORC ColumnVectorBatch, respectively) before further query execution can begin. On the other hand, Parquet exposes a *streaming API* that allows pipelining data parsing and query execution, leading to more optimization opportunities. However, Parquet does not itself provide any dedicated in-memory data structures.

4 Methodology

In the subsequent sections, we benchmark the performance of Arrow, Parquet, and ORC over relational data. This is not strictly an apples-to-apples comparison because each format was developed with a different use case in mind: Arrow eases the sharing of in-memory data across systems, Parquet is a generic on-disk format, and ORC is a storage format for relational big data systems. Nevertheless, this comparison is important for evaluating the design choices (e.g., encoding method, compression, implementation decisions) made by each format and to understand the limitations and opportunities when using these formats in analytical DBMSs.

Dimensions

To most fairly compare the formats, we evaluate each format across the following dimensions:

1. *Compression ratio* Each format applies different encoding methods and supports different compression algorithms. The final achievable compression ratio is a result of these decisions, and so we evaluate each format using the variously supported encoding and compression algorithms (Sect. 5.1).
2. *Transcoding throughput* While compression ratio alone is sufficient if we care only about minimizing disk or memory usage, this comes at the cost of having to compress, convert, and decompress (i.e., transcode) the data when accessing it (Sect. 5.2).
3. *Data access* For each data type, what are the costs of accessing them? Data is often accessed by column (i.e., projected) or filtered using a predicate. We evaluate the costs of each format when applying simple data access operations (Sect. 6).
4. *End-to-end evaluation over subexpressions* Since we care about the performance of each format when evaluating analytical queries, we explore the performance of each format over a set of query subexpressions drawn from TPC-DS (Sect. 7).
5. *Advanced features* Given the many trade-offs baked into each format, we explore the extent to which we can extend them to support novel features such as computation push-down into the encoded domain and hardware acceleration (Sect. 8).

To summarize our findings, in Table 4 we show the structure of our experiments and the overall best format for each dimension.

Setup

All experiments are performed on an Azure Standard D8s v3 (8 vCPUs, 32 GiB memory), premium SSD LRS, and Ubuntu 18.04. We test Apache Arrow 5.0.0, ORC 1.7.2, the Apache Parquet Java API version 1.9.0, and PyArrow 17.0.0. Where needed, we use the Apache Arrow C++ library to

Table 4 Evaluation overview and key results

Evaluation dimension	Best overall	Key advantage	Section
<i>Compression ratio</i>	Parquet	Comprehensive encoding and compression support	5.1
<i>Compression throughput</i>	Arrow feather	Fast serialization	5.2.1
<i>Decompression throughput</i>	Arrow feather	Fast deserialization	5.2.2
<i>Vector transcoding</i>	Arrow feather	Fast (de)serialization	5.2.3
<i>Projection evaluation</i>	Parquet, ORC	Fine-grained skipping while loading data	6.1
<i>Predicate evaluation</i>	ORC	Fine-grained loading, dedicated in-memory representation	6.2
<i>Bitmap evaluation</i>	ORC	Fine-grained loading, dedicated in-memory representation	6.2.3
<i>Subexpression evaluation</i>	ORC	Fine-grained loading, dedicated in-memory representation and efficient skipping	7
<i>Direct querying</i>	Parquet	In-memory mapping with data skipping and direct querying	8
<i>Vectorized execution</i>	Parquet	In-memory mapping with data skipping, direct querying, SIMD	8

write in-memory Arrow tables to disk. We perform experiments using (i) the TPC-DS dataset at scale 10, (ii) the Join Order Benchmark (JOB) [48], (iii) the Public BI Benchmark (BI) [49], (iv) real-world datasets drawn from public data sources including GIS, machine learning, and financial datasets (CodecDB) [25], and (v) popular retrieval augmented generation (RAG) and embedding-related datasets drawn from the Hugging Face community-curated collection [50].

For all the experiments, we report numbers when the system caches are cold by default. For selected experiments, we also report numbers when caches have been warmed up, i.e., to simulate frequently accessed datasets. Unless stated otherwise, we use each format's default settings. While different configurations could affect the results, data format access logic—including (de)serialization flow, (un)compression, and encoding/decoding control flow behind data access APIs—generally has a greater impact on query performance. Moreover, users often stick to default configurations, and exploring a broader parameter space would significantly expand an already large set of experiments, potentially obscuring the core comparison. We leave fine-grained configuration tuning as future work.

5 Compression and transcoding

In this section, we evaluate the compression performance of Arrow, Parquet, and ORC (Sect. 5.1) and the related costs for transcoding data from compressed to in-memory formats (Sect. 5.2).

Table 5 Total size (in GB) by format for columns in the CodecDB, BI, and JOB datasets

Data Type	# Cols.	Raw Size	Parquet Size	ORC Size	Arrow Size	Arrow (DICT)
Integer	12k	57.3	9.8	13.5	59.3	59.3*
Float	7k	58.8	24.0	58.2	59.8	59.8*
String	13k	373.5	31.0	62.2	403.4	118.3
Total	31k	489.7	64.7	133.9	522.5	237.4
Compression ratio			0.13	0.27	1.07	0.48

We serialize each column separately into each format and group their compressed size by data type

The raw dataset is in CSV format

For Arrow, we report with dictionary encoding enabled (Arrow DICT) and disabled (the default)

We copy the file size (marked with *) from the Arrow default column for CR computation as there is no dictionary support for integer and float types

5.1 Encoding & compression performance

We begin by exploring the compression performance of each format through three sets of experiments. In the first experiment (Sect. 5.1.1) we evaluate how each format's supported encodings perform over a set of real-world datasets. The last two experiments leverage TPC-DS [51] to illustrate the performance of each format when compression is applied on top of encodings. For the synthetic experiments on TPC-DS, we begin by evaluating compression algorithms over the full dataset (Sect. 5.1.2), and then explore how compression performance varies by data type (Sect. 5.1.3). Finally, we assess the compression performance of each format on real-world embedding datasets (Sect. 5.1.4).

Table 6 Average and stddev compression ratios on each dataset by data type

Dataset	Type	Parquet		ORC		Arrow		Arrow dict	
		AVG	STD	AVG	STD	AVG	STD	AVG	STD
CODEC	String	0.256	0.359	0.258	0.31	1.585	0.692	0.99	0.889
	Float	0.351	0.294	1.27	1.099	1.383	1.286		
	Integer	0.291	0.27	0.273	0.16	1.241	0.964		
BI	String	0.074	0.148	0.119	0.227	1.391	0.474	0.699	0.538
	Float	0.335	0.217	1.558	0.886	1.588	0.884		
	Integer	0.2	0.162	0.249	0.173	1.567	0.522		
JOB	String	0.872	1.741	0.923	1.227	2.607	3.316	2.557	4.791
	Integer	0.713	2.186	0.432	1.038	1.687	4.485		
ALL	String	0.207	0.343	0.222	0.307	1.535	0.677	0.915	0.871
	Float	0.342	0.255	1.427	0.999	1.494	1.09		
	Integer	0.247	0.267	0.261	0.18	1.408	0.837		

5.1.1 Encoding performance over real-world datasets

In this experiment, we group each data column by data type, convert each column one-by-one into each format, and finally aggregate the statistics of the compressed columns. Table 5 shows the overall compression performance and statistics, grouped by data type, over the ~31k columns in the CodecDB, Public BI and JOB, datasets. Table 6 provides a detailed view of the compression ratio for each dataset and data type. To avoid confusion, we do not apply any further compression after default encoding techniques are applied (we will explore how each format behaves when compression is enabled in the following sections).

We begin by showing detailed CDFs over the ratio of the number of distinct values to number of distinct rows (Fig. 4). As we can see, each dataset has a distinct CDF shape, confirming that our analysis covers data with varying characteristics. Next, Fig. 5 shows the compression ratio (CR) performance on the CodecDB real-world dataset. For integers in this dataset (Fig. 5a, dictionary+RLE encoding in Parquet performs well for low distinctness ratios, while RLE alone in ORC works relatively well for high distinctness ratios for integer columns. This is the reason there is a crossover point between Parquet and ORC. For floats (Fig. 5b, Parquet outperforms both ORC and Arrow Feather due to its use of dictionary encoding. These formats perform similarly as they both use a plain encoding. For strings (Fig. 5c, both Parquet and ORC fall back to plain encoding on some columns when dictionary encoding takes up more space relative to plain encoding. But the two formats work differently in this case—Parquet’s plain encoding introduces a higher space cost for saving the string length values, whereas ORC’s plain encoding uses RLE for string length values. However, Parquet’s dictionary encoding is more effective than ORC because of the extra layer of RLE for the dictionary-encoded keys. That is why Parquet works bet-

ter in terms of total compressed size while ORC works better in terms of the number of effective compression ($CR < 1$) on the CodecDB dataset.

Fig. 6 shows compression performance on the BI dataset. In this datasets we observe that both the integer and string columns have a lower distinctness ratio than did CodecDB, so Parquet’s dictionary encoding has better compression performance in this figure. This is also seen in Table 6. We next show compression performance over the JOB dataset in Fig. 7. For integers (Fig. 7a, the Arrow line increases significantly at around 0.5 because Arrow uses plain encoding for integer values. The original text values of many integer columns (with millions of rows) take 8 bytes on average (e.g., "4167491\n"), while the plain encoded integer values consume 4 bytes, resulting in ~0.5 compression ratio. String compression (Fig. 7b does not exhibit this behavior.

We finally show the compression ratio CDFs for each data type in Fig. 8 where we focus on the effective compression ratio range [0, 1]. Overall, Parquet exhibits the best compression performance due to its efficient encoding methods. Figure 9 further illustrates the CDFs for each data type, taking into account the number of distinct values in each column of all the datasets. This analysis highlights the impact of value distinctiveness on compression performance, particularly for formats that rely on dictionary encodings.

Discussion Overall, as we can see in Table 5, Parquet performs the best over the whole dataset and is able to reduce the size of the column data to about 13% of the original. ORC is able to compress the dataset to ~27%. By contrast, Arrow Feather—with default settings—exhibits a 7% increase in size compared with the raw text file. We found that this overhead is introduced by the format’s metadata, which adds a four-byte length prefix to each variable binary entry (i.e., the string “abc” consumes seven bytes in total). It also pads numerical data types. On the other hand, with DICT enabled, Arrow Feather compresses string columns by 68% and the

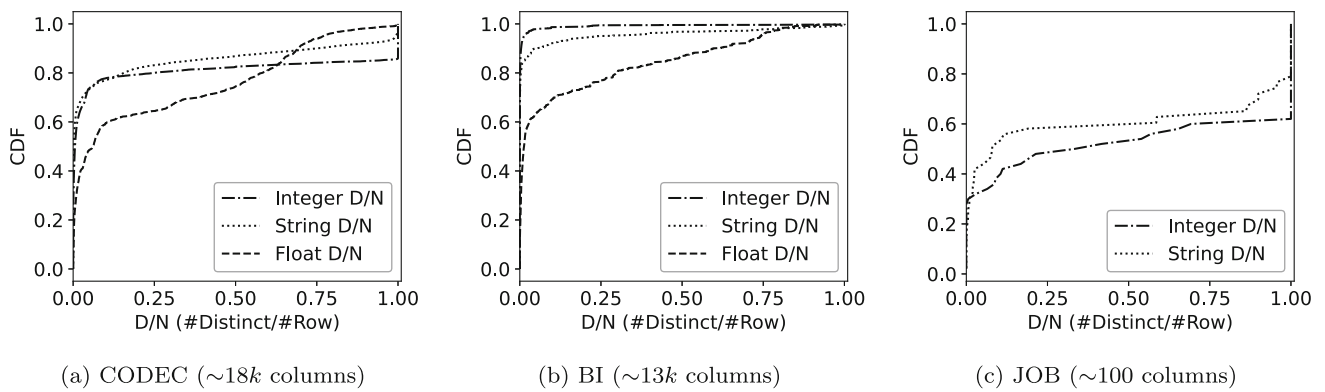


Fig. 4 Ratio of number of distinct values (#Distinct) to the number of rows (#Rows) in the CodecDB, Public BI and JOB datasets. The spikes for integer types near $D/N = 1$ in the CODEC and JOB datasets occur because of primary key columns, which contain no duplicate values

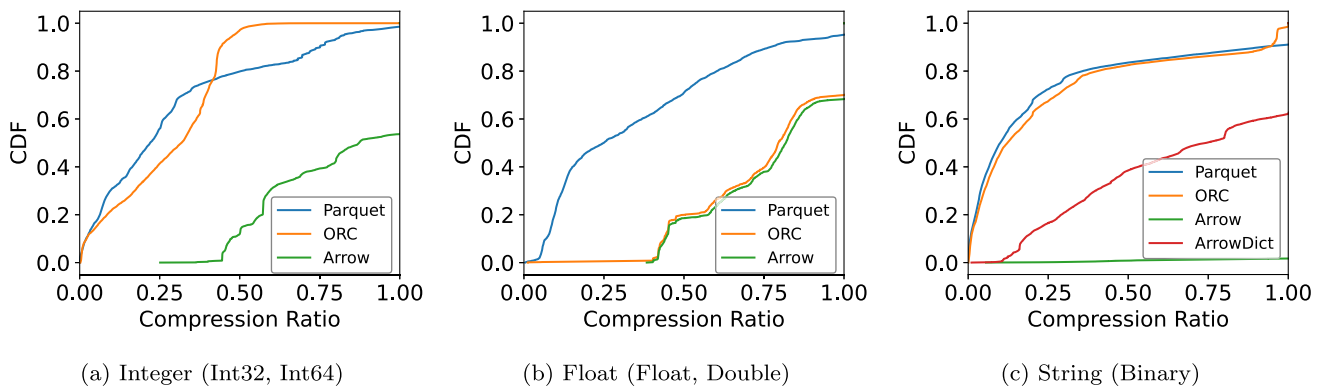


Fig. 5 Compression ratios on the CodecDB real-world datasets with $\sim 18k$ columns. The figures show the effective compression ratio (CR) in the range $[0, 1]$. The CDF lines do not start near zero, as they are bounded by compression performance. For example, Arrow's minimal CR in Fig. 5a is around 0.25 because of its lacking compression support,

whereas other formats can start near (but not at) zero due to their more effective compression. On the other hand, the CDF lines do not always reach 1.0 at $CR = 1.0$ because of underperforming compression on some columns

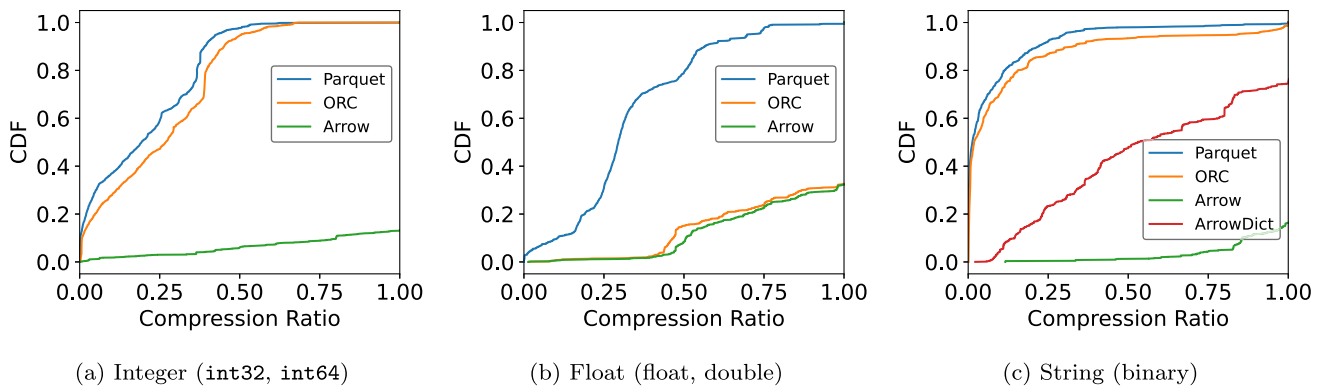


Fig. 6 Compression ratio performance on the BI dataset over $\sim 13k$ columns

Fig. 7 Compression ratio performance on the JOB dataset with approximately ~100 columns. (There are no floating-point columns in this dataset.)

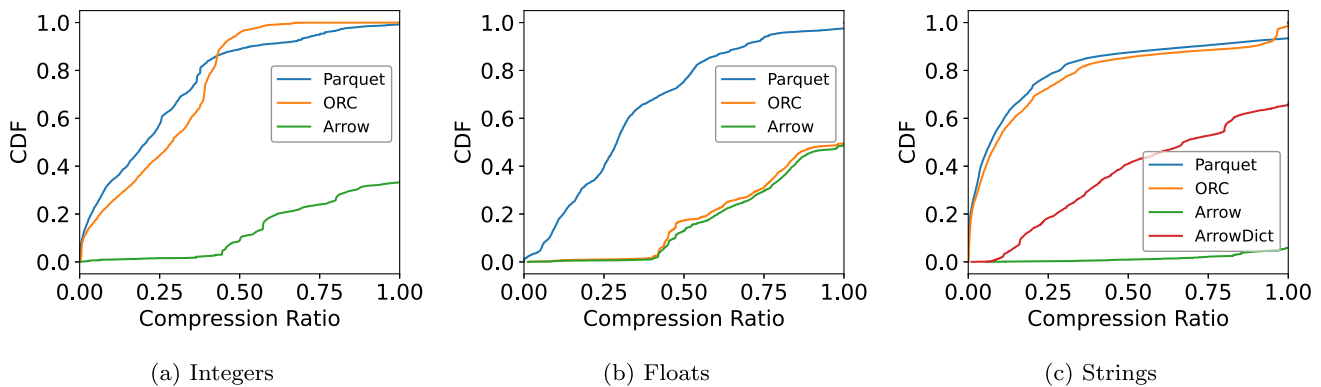
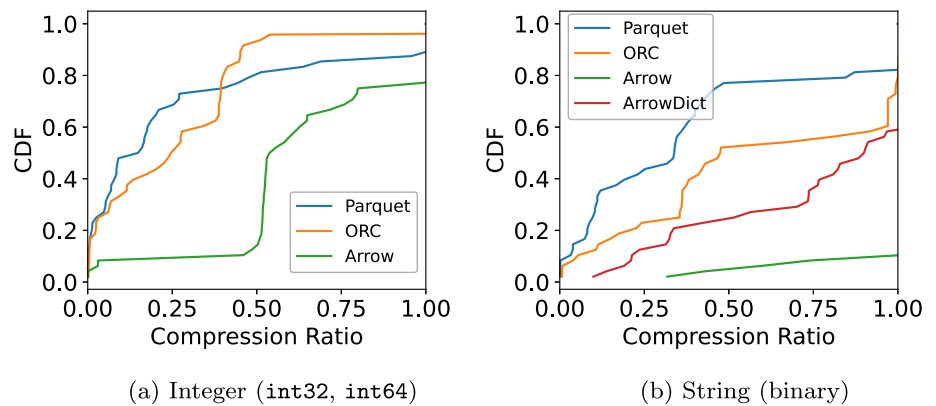


Fig. 8 Column compression ratio CDFs over the CodecDB, BI and JOB datasets

whole dataset by 52%. This is further supported by Fig. 6c, where the string columns in the BI dataset have a lower distinctness ratio than those in the CodecDB dataset. On the other hand, string columns in the JOB dataset have a relatively high number of distinct values, and thus dictionary encoding (i.e., Parquet and ORC) performs relatively poorly (also shown in Table 6).

For integers, ORC exhibits varying compression performance relative to Parquet. ORC achieves a better compression ratio for the CodecDB and JOB datasets (which contain a relatively higher number of distinct values), while it is worse for the BI dataset (which has a lower number of distinct values). This is because ORC applies RLE for integer columns (see Table 3), which performs better for columns with fewer distinct values, whereas Parquet applies DICT-RLE, which is slightly worse. Because of this, we observe a crossover point for the Parquet and ORC CDFs in Fig. 8a.

For floats, as we can see from Fig. 8b, Parquet outperforms ORC and Arrow Feather because of dictionary encoding. ORC and Arrow Feather perform similarly as they both use plain encoding. For strings, Parquet and ORC outperform Arrow. Interestingly, both Parquet and ORC fall back to plain encoding on some columns when dictionary encoding takes up larger space than plain encoding, but their dictionary encoding work differently: Parquet's plain encoding

introduces a higher space cost for saving the string length values, while ORC's plain encoding uses RLE for string length values. However, Parquet's dictionary encoding is more effective than ORC because of the extra layer of RLE for the dictionary-encoded keys. That is why Parquet works better in terms of total compressed size (see Tables 5 and 6) while ORC works better in terms of the effectiveness (compression ratio < 1; see Fig. 8c).

5.1.2 Compression performance

For this experiment, we report the compression ratio of each format on the full TPC-DS dataset when different compression algorithms are applied. We evaluate Zstandard (Zstd) at level 1 (we evaluate other levels later in this section), LZ4, Gzip, Snappy, and Zlib compression algorithms, and compare them against an uncompressed variant where data is encoded using the default settings. The results of this experiment are shown in Fig. 10. In the uncompressed case, Parquet is about $2\times$ better than Arrow Feather because Arrow Feather does not apply any encoding. However, when compression is enabled, Arrow performs within $\sim 30\%$ of Parquet. ORC achieves a similar compression ratio as Parquet, except under LZ4. In this case, ORC automatically disables compression

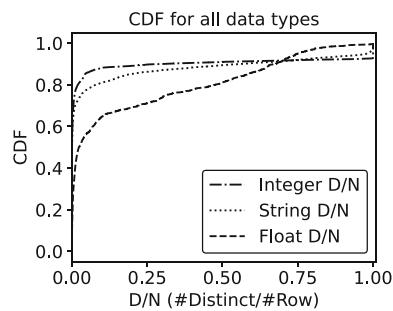


Fig. 9 Distinct value CDFs over all datasets

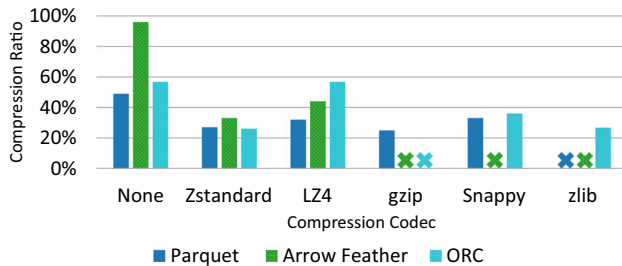


Fig. 10 Compression ratio (compressed size / original CSV size) on TPC-DS (smaller is better). Uncompressed (None in the figure) only encodes using default settings. Not all formats support all compression algorithms

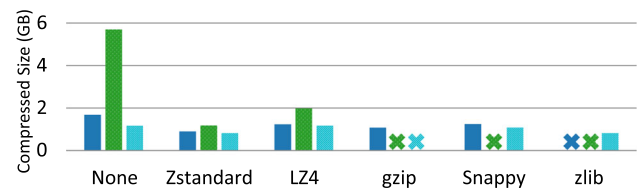
because it detects that the LZ4 compressed data size is greater than the original data size.

Finally, we observe that different compression algorithms yield different compression ratios. For example, increasing Zstd's level from 5 to 9 yields more aggressive compression and achieves smaller sizes. However, this gain is minimal ($< 1.5\%$) while the compression time increases by $\sim 3\times$ for Arrow Feather and $\sim 2\times$ for Parquet. We will show in Sect. 5.2 how decompression costs are also impacted by the choice of compression algorithm.

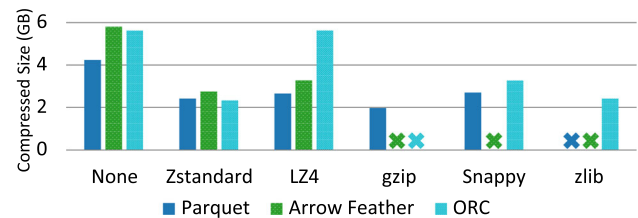
5.1.3 Compression performance by data type

In this experiment, we look at the performance over various column types in the TPC-DS dataset. Specifically, we evaluate compression performance on integer (both `int32` and `int64`), double, and string data types. We extract all columns of a given data type, compress them, and report the aggregate sizes by type. The results are in Figures 11 and 12.

First, consider Fig. 11a which shows the aggregate compression performance on the integer columns. ORC achieves slightly better compression performance than Parquet. This is because Parquet applies DICT and switches to plain encoding for some of the columns, whereas ORC always applies RLE. Arrow Feather does not encode by default. This leads to the worst compression ratio when compression is disabled. Nevertheless, all three data formats perform similarly



(a) Integers (`int64` and `int32`)



(b) Doubles

Fig. 11 Total size on disk after compressing the numeric columns in TPC-DS

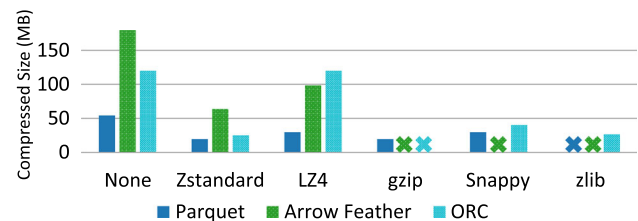


Fig. 12 Total size on disk after compressing the string columns in TPC-DS

when compression is enabled, except for LZ4, where Arrow Feather is almost 50% worse because it lacks encoding support for integers (we observe a similar result in Fig. 10).

Next, Fig. 11b shows the aggregate compression performance for the double columns. Parquet also applies DICT to this data type, whereas ORC and Arrow Feather do not encode at all. Because of this, Arrow and ORC have very similar performance both in the uncompressed and compressed setting, whereas Parquet is slightly better. The ORC outlier for LZ4 happens for the same reason as discussed in Sect. 5.1.2.

Finally, Fig. 12 shows compression performance on string columns (both variable- and fixed-length). By default, Arrow does not encode this type, whereas ORC and Parquet apply DICT. Among all formats, Parquet has the best compression performance, followed by ORC and Arrow. ORC produces larger compressed sizes than Parquet, because: (i) ORC has a smaller default block size and thus pays more dictionary overhead per row batch; and (ii) it more frequently falls back to plain encoding because of its row batch-level dictionary encoding (versus the chunk-level used in Parquet). Again, LZ4 ORC disables compression because it offers no benefit.

5.1.4 Compression performance over embedding datasets

We next explore compression performance of each format on vector-oriented datasets. To do so, we used the Hugging Face Dataset portal [50] as a source of retrieval-augmented generation (RAG) and other embedding datasets. For the RAG datasets, to evaluate different scales, we selected two high-performance embedding models: *UAE-Large-V1* [52] and *SFR-Embedding-Mistral* [53], with respective parameter sizes of 335 and 7100 million. We additionally assembled a diverse set of 38 unique embedding datasets having at least 500 monthly downloads. These datasets feature dimensions ranging from 256 to 4096, and record counts ranging from thousands to millions.

We load these datasets into in-memory NumPy arrays and subsequently utilize PyArrow [54] to convert them into Parquet, Arrow Feather, and ORC. We then apply compression as described in the previous section.

We evaluate two data layouts for the vectors. First, we decompose each n -dimensional vector into n float values and store each value in a separate column (i.e., a *columnar layout*). Second, we persist each vector into a single nested column containing n sub-elements (i.e., a *nested layout*; see Table 3).

Figures 13 and 14 respectively show results for the columnar and nested layouts. Considering that embeddings comprise high-dimensional and high-entropy vectors, we see distinct behaviors across formats. For the columnar layout, Parquet struggles with the large number of columns, rendering its dictionary encoding less effective compared to the plain encoding used by Arrow Feather and ORC. Although ORC's Int-RLE encoding typically works well for low-precision values, it does not outperform Arrow's plain encoding due to the high-precision embedding values. Apart from differences with LZ4 compression, Arrow Feather and ORC show similar performance, reflecting the same reasons discussed in Sect. 5.1.2.

For the nested layout (Fig. 14), neither format effectively compresses the embedding dataset in the uncompressed setting, so their default vector encodings display minimal differences on such high-dimensional, high-entropy, and high-precision vectors. Even with compression enabled, this pattern mostly persists, except when using Zstd—where ORC performs slightly better (less than a 10% improvement), which can be attributed to ORC's reliance on a different version of the Zstd compression library. ORC's LZ4 remains ineffective for the same reasons discussed in Sect. 5.1.2.

Discussion In summary, current formats do not adequately handle compression for widely used embedding datasets. Surprisingly, when we compare each format's uncompressed representation (the “None” category in Fig. 14) to the uncompressed NumPy baseline (the orange line), we find that all formats produce *larger outputs* than this baseline. This is

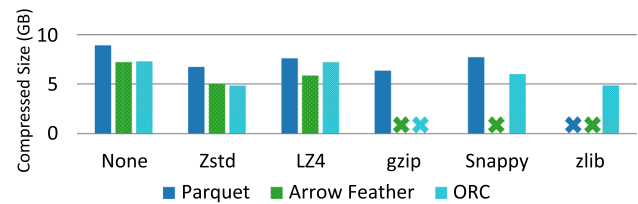


Fig. 13 Total size on disk after compressing embedding datasets using columnar layout (i.e., each n -dimensional vector is mapped into n columns). The CSV baseline (not shown in figure) consumes 18 GB

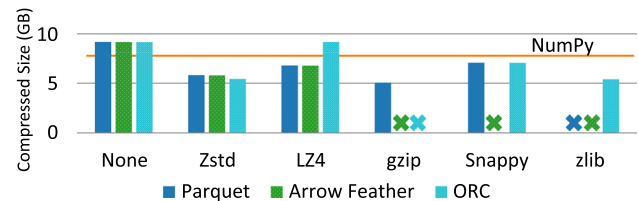


Fig. 14 Total size on disk after compressing embedding datasets using the nested layout (i.e., each vector is serialized as a single nested column). The “NumPy” baseline shows the size of data serialized using the NumPy uncompressed standard binary format

due to overhead introduced by the nested layout encodings and the hierarchical metadata inherent to each format.

When comparing compressed size over both the columnar and nested layouts, Arrow and ORC produce larger files with the nested layout because of their additional per-record, metadata-related bits. On the other hand, Parquet produces smaller file sizes with the nested layout because its dictionary encoding is less effective on float types under the columnar layout.

In addition to the high-dimensional and high-entropy nature of embedding vectors, their high-precision further limits compression performance across all formats. This raises a pertinent question: is such high precision truly necessary for downstream AI tasks, especially those relying on high-dimensional distance calculations like RAG and k NN? Liu [35] addresses this by evaluating a 1NN task on the UCR dataset collection, using various precision levels for the input float numbers. They report the 1NN accuracy for all 128 datasets, normalizing each dataset's accuracy by its corresponding full-precision accuracy. The results indicate that relative accuracy decreases gracefully as precision diminishes. Nonetheless, the performance remains remarkable, demonstrating the potential for reduced precision in machine learning tasks. This opens up the possibility that precision-controlled encoding, such as BUFF [26], could be a suitable option for embedding encoding in columnar formats.

5.2 Transcoding overhead

In practice, storage formats are converted into (or from) an in-memory presentation on reads (writes). We now evaluate

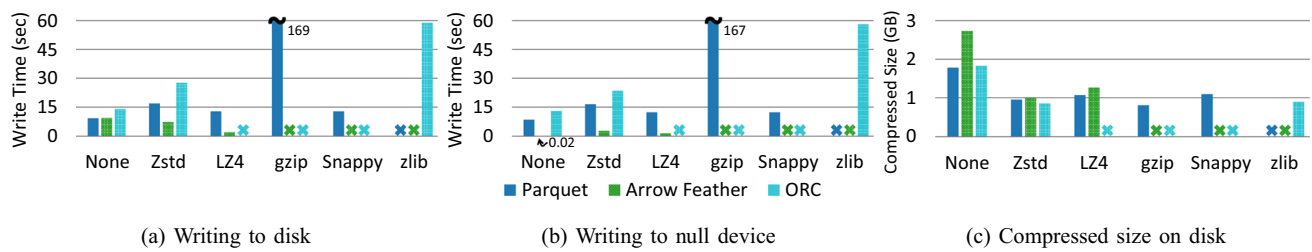


Fig. 15 Write time from an Arrow in-memory table to each format stored either on disk or in memory

the overheads in transcoding (i.e., decompressing, converting, and compressing) each format. Specifically, Sect. 5.2.1 explores the time required to compress and serialize each format from a common in-memory representation, while Sect. 5.2.2 evaluates the overhead of loading data, i.e., deserializing and decompressing each format into an in-memory representation amenable to query execution. In Sect. 5.2.3, we finally evaluate both the serialization and deserialization cost over embedding datasets.

5.2.1 Compression overhead

Our first experiment in this section explores how long it takes to serialize (and compress) the data from an in-memory representation to each disk format. For this experiment we use the `catalog_sales` TPC-DS table. The `catalog_sales` is a large (~14M rows) and wide (34 columns) table containing integers and doubles. Its raw data size is 3GB. All formats support serializing from an Arrow Table, and so we adopt it as our common in-memory representation.

Figure 15 shows the runtimes when: (i) writing to disk (15a); (ii) writing to the null device, which avoids any I/O overhead (15b; as well as (iii) the data sizes per format (15c. We omit the LZ4 and Snappy bars for ORC as the Apache Arrow C++ library has limited compression support for the ORC format. Starting with Fig. 15a, we can see that Arrow Feather is the most efficient format in terms of compression and serialization runtime because it does not encode data. On the other hand, Arrow Feather's lack of encoding leads to almost a 50% larger footprint on disk (Fig. 15c. Interestingly, ORC compression time is 50% slower than Parquet with comparable or slightly better compression ratio on disk (up to 15% better). We think that this is because of better Parquet support in Arrow; both projects share the same codebase and data structures.

Finally, we isolate the compression overhead in Fig. 15b by avoiding disk I/O by writing to the null device. Here we can see a decrease in runtime for all the formats, although of different magnitudes. Arrow Feather has the biggest difference, thanks to its inherent zero-copy implementation in Arrow. The compression time for Parquet and ORC does not

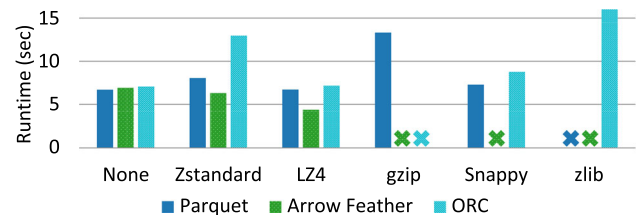


Fig. 16 Runtime (in seconds) for decompressing the TPC-DS `catalog_sales` table from the on-disk formats into in-memory Arrow

change substantially because the encoding and compression operations dominate the total runtime.

5.2.2 Decompression overhead (i.e., table scan)

In this experiment we investigate the overhead of loading the `catalog_sales` TPC-DS table from disk into memory. Our goal with this experiment is to simulate the overheads involved when a query processor is required to load and transform a compressed dataset into a plain in-memory format amenable to query execution. We start from data on disk in the Parquet, ORC, or Arrow Feather formats, and we report the time required to load the data and convert it into the Arrow in-memory format. The results are shown in Fig. 16.

Interestingly, loading compressed data has 30% less overhead than the uncompressed case for Arrow under LZ4. This is because LZ4 requires less disk I/O (since the file on disk is smaller; see Fig. 15c while also providing “fast enough” decompression relative to the other compression methods. For the other cases, Arrow always exhibits the best performance: this is expected since it does not require decoding the data and its on-disk compressed size is reasonable. Parquet is slightly worse than Arrow because of the cost of decoding data, while ORC has the worst performance (it is particularly bad for Zstd and zlib). We think that this is due to decompression settings such as block size, buffer size, etc. In general, for formats that heavily leverage encoding (i.e., Parquet and ORC) data compression leads to a heavy penalty on read performance.

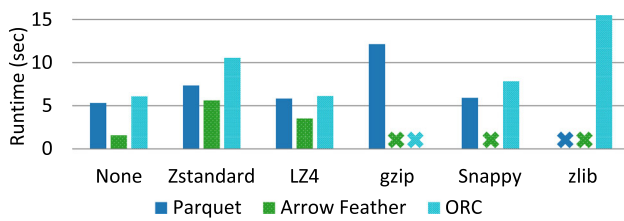


Fig. 17 Runtime (in seconds) for decompressing the TPC-DS catalog_sales table from the formats in memory (ramdisk) into in-memory Arrow

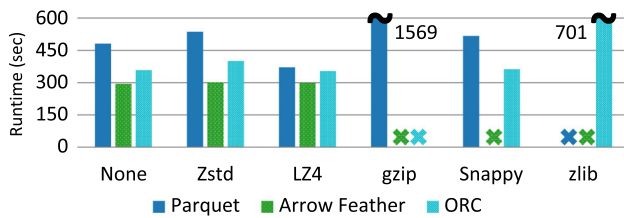


Fig. 18 Write time using a columnar layout (i.e., n -dimensional vectors are represented as n columns) to each format on disk. The CSV baseline (not shown in figure) took 1,982 s to write

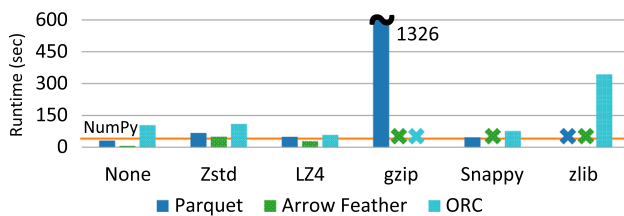


Fig. 19 Write time for vectors using the nested layout to each format on disk. The “NumPy” line shows time to written vectors serialized using the NumPy uncompressed standard binary format

To isolate disk I/O from compression overheads, we load each compressed dataset onto a memory-resident disk mounted on `tmpfs`. As we can see from Fig. 17, in all cases the runtimes decrease, especially for Arrow without compression. This result is intuitive because, for uncompressed data, the data size is much larger and disk bandwidth is saturated. Conversely, decompression is CPU-bound and not substantially impacted by the cost of bringing data into memory. Combined with previous compression experiments in Fig. 15, this shows the benefits of Arrow as a fast inter-process format, when disk I/O and size are not the bottleneck.

5.2.3 Transcoding cost over embedding dataset

We next evaluate the compression times using the columnar and nested layouts described in Sect. 5.1.4. The respective results are shown in Fig. 18 and Fig. 19. ORC and Arrow Feather, which utilize plain encoding, demonstrate greater efficiency compared to Parquet’s dictionary encoding. This results in shorter compression times under the columnar layout. Arrow Feather consistently outperforms the others due to

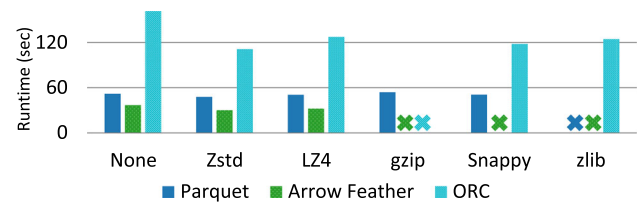


Fig. 20 Runtime (in seconds) for decompressing the table from the on-disk formats into an in-memory table using the columnar layout

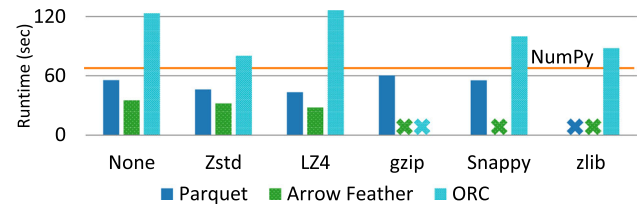


Fig. 21 Runtime (in seconds) for decompressing the table from the on-disk formats into an in-memory table using the nested layout. The “NumPy” line shows time to decompress data serialized using the NumPy uncompressed standard binary format

its more efficient serialization. For the nested layout, where all formats must encode and compress vector embeddings, Parquet’s Dremel-based encoding surpasses ORC’s encoding in efficiency. Arrow remains the top performer overall, benefiting from its native serialization efficiency. When comparing the columnar and nested layouts between the same format, each format shows better efficiency with the nested layout. This is due to more specialized encoding and the simplified writing logic when emitting a single nested column.

We next evaluate the performance of loading embedding datasets from disk into memory. The datasets are scanned into in-memory NumPy arrays, and we record the scan time for each file. We then report the runtime for loading the data in two layouts (columnar and nested) and respectively show the results in Figs. 20 and 21.

Arrow Feather consistently outperforms the other formats in both cases, whereas ORC shows the poorest performance. Specifically, the deserialization time for ORC is typically 2–3 times longer than for Parquet and 3–4 times longer than for Arrow Feather. The nested layout generally requires more load time due to more complex reading logic, which involves checking the definition and repetition levels for decoding the Parquet and Arrow and referring to the present and length streams for ORC. By contrast, the columnar layout benefits from bulk-reading logic and greater parallelism, since the reads can be distributed across multiple column readers.

In our comparison of vector embedding serialization and deserialization performance across these formats, it is evident that all formats fall short of the NumPy data science format in terms of write efficiency. This is notable despite the generally poor compression performance observed in these formats. Among them, Arrow Feather distinguishes itself with its

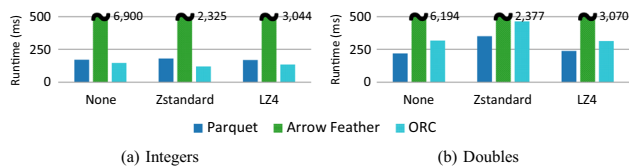


Fig. 22 Projecting numeric types on the `catalog_sales` table

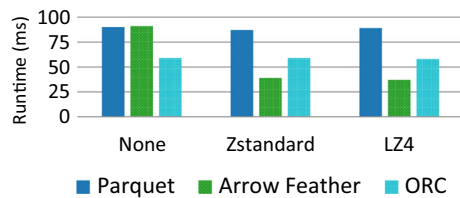


Fig. 23 Projecting strings on the `customer_demographic` table

superior read performance relative to NumPy, attributed to its more efficient serialization capabilities. However, it still lacks robust encoding support. Parquet, on the other hand, demonstrates read performance comparable to that of NumPy. In stark contrast, ORC consistently exhibits the poorest performance for both reading and writing operations. These findings underscore the importance for open columnar formats to better support machine learning use cases. Introducing new encodings that effectively and efficiently handle high-dimensional, high-entropy, and high-precision embedding vectors in Parquet appears to be a feasible near-term solution. However, compressing high-entropy data remains challenging due to information theory constraints. Until such encodings become available, Arrow remains the best overall choice for embedding workloads. We also see emerging formats like Lance [55] and Vortex [56], which build on Arrow, enabling additional compression. Moving forward, improving these formats through more efficient encoding techniques and better-optimized serialization/deserialization processes will be key to meeting the ever-evolving demands of modern data-centric workflows.

To summarize, encoding and compression choices greatly impact performance, with formats like Parquet and ORC targeting size on disk, while Arrow targets raw read performance. To optimize both size and performance, formats should be carefully tuned to the workload and use case, and workload-aware compression selection is crucial. It remains an open question of how much computation can be pushed into the encoded space to minimize the decoding step while maximizing the compression ratio.

6 Data access microbenchmarks

Having explored the overheads associated with encoding, compression, and scan operations, we next evaluate

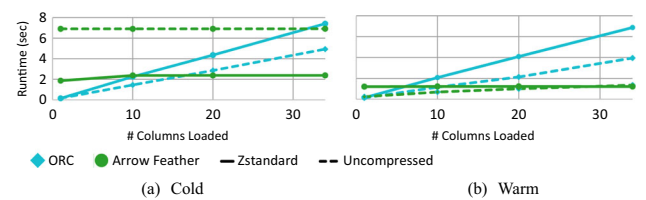


Fig. 24 Profiling single column to full table sequential loading for ORC and Apache Arrow from the `catalog_sales` table

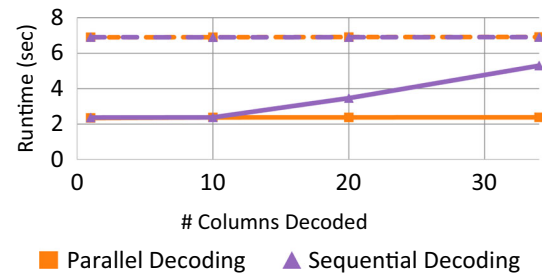


Fig. 25 Arrow serial vs parallel (default)

the performance of accessing data in the context of common relational operations found near the leaves of a query plan. Specifically, we explore the performance of projecting columns in a dataset (Sect. 6.1) and applying filters (Sect. 6.2). In this and subsequent sections we only consider Zstd and LZ4 compression since we evaluated the trade-offs of the other compression algorithms in Sect. 5.

6.1 Projection

We first explore projection performance for common data types. To do so, we: (i) load the data from disk, (ii) decompress and decode the projected columns, and (iii) convert to in-memory Arrow.

Figures 22a and 22b show the respective runtimes for projecting an integer and double column in the `catalog_sales` table of TPC-DS. Since `catalog_sales` does not contain any string columns, Fig. 23 instead shows the runtime of projecting a string column drawn from the `customer_demographic` table. This table is narrower than `catalog_sales` and has 9 integer and string columns. The raw data size is 80 MB with ~2 million rows.

ORC is the most performant format for projections over integer columns because it applies RLE encoding, which yields a higher compression ratio and lower I/O costs. Parquet is slightly slower as it leverages DICT, which (i) inflates the compressed file size due to dictionary storage overhead; and (ii) slows down the loading process by introducing dictionary lookup overhead. Arrow Feather is by far the worst in this experiment since its API requires parsing the entire byte-array including all columns (not just the projected sub-

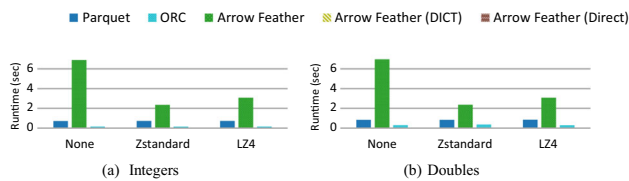


Fig. 26 Filtering numeric types on the `catalog_sales` table

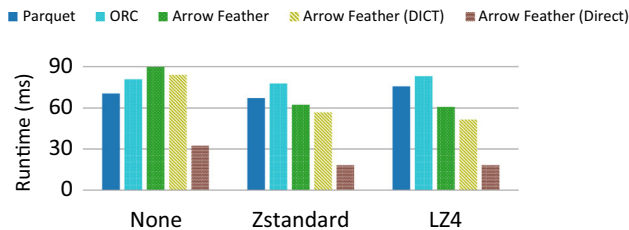


Fig. 27 Filtering strings on the `customer_demographic` table

set) from disk before column chunks loading can commence (we further discuss this in Sect. 6.1.1).

For doubles, Parquet offers the best performance, as we can see from Fig. 22b. In this case Parquet's DICT encoding leads to a much smaller compressed size in this dataset (which has low cardinality; see Sect. 2.2). ORC does not encode doubles and is thus slightly slower than Parquet. Arrow Feather lags far behind for the same reason it did with integer columns.

Figure 23 shows the results of projecting a string column.¹ Despite loading all columns into memory before projecting, Arrow Feather outperforms the other formats in this experiment. This is because by default Arrow Feather does not dictionary encode its data and is therefore able to entirely avoid the associated lookup overhead. ORC performs slightly worse than Arrow Feather because ORC separately RLE-encodes each string's lengths, introducing additional decoding overhead. Parquet, however, performs the worst because its application of DICT encoding inflates projection time relative to the modest I/O cost of loading the `customer_demographic` table. Finally, we want to point out that ORC is also faster than Parquet because of its API, which allows for efficiently transforming data into its dedicated in-memory representation, whereas Parquet deserializes data into memory using its relatively slow streaming style API with rudimentary data access control.

¹ The runtimes between the numeric and string experiments are not directly comparable since `catalog_sales` has an order of magnitude more records (and $\sim 5\times$ average row size in bytes) than does `customer_demographic`. Nevertheless, `customer_demographic` is the largest table in TPC-DS that contains a string column.

6.1.1 Profiling data loading

Our previous experiments show that Arrow Feather loading is far more expensive than ORC and Parquet. We observed that Arrow Feather, even when projecting a single column, requires parsing the entire byte-array. To better understand these trade-offs, we now explore Arrow's data loading code in deeper depth and contrast it with ORC.

We begin by evaluating the cost of loading a single column against the cost of loading the whole table. The results are shown in Fig. 24. Overall, ORC performs best when extracting a single column. Relative to Arrow Feather, this occurs for several reasons. First, ORC offers the ability to perform fine-grained reads at the column level while Arrow Feather requires reading, decompressing and decoding the entire row batch before projecting the target column(s). This means that ORC's runtime is proportional to the number of columns extracted, while extracting one single column from an Arrow Feather file is only $2\times$ faster than extracting the full table. Second, upon examining the Arrow Feather deserialization logic, we observed that it suffers from substantial synchronization overhead when parsing column chunks within each row batch. In particular, we found that the lock acquisition step consumed $\sim 80\%$ of the runtime for each row batch.

To better understand synchronization issues, we finally evaluate Arrow Feather table loading using its native data loading API in both sequential and parallel execution modes. Parallel loading mode leverages a global CPU thread pool to parallelize column decompression. This potentially results in a performance advantage relative to ORC, which serially decompresses columns. We see this improvement in Fig. 25 where there is a large difference between the two modes when compression is enabled (the Zstd lines in Fig. 25) and no difference when compression is disabled.

6.2 Filtering

In this section we evaluate each format's performance when applying filter operations. We separately consider predicate evaluation (Sects. 6.2.1 and 6.2.2 respectively for numeric and string columns) and bit-vector evaluation (Sect. 6.2.3).

6.2.1 Numeric predicates

We evaluate two predicates over the `customer_sale` table (with respective selectivities of 65% and 30%):

```
CS_SHIP_DATE_SK > n (integer column filter)
CS_WHOLESALE_COST > n (double column filter)
```

For each format, we load the data from disk, decode the target column to its in-memory representation when available (see Sect. 3), and evaluate the predicate to generate a

bit-vector x (i.e., entry $x_i = \top$ when row i matches the predicate). For Parquet, which does not have a dedicated in-memory representation, we use its native API and interleave decompression with predicate evaluation.

The results for each predicate are shown respectively in Figs. 26a and 26b. The trends are similar: overall, ORC outperforms both Parquet and Arrow Feather for each data type and compression scheme. Arrow Feather's performance is 3–4 $\times$ worse than Parquet in the compressed case, but when uncompressed it further lags (to more than 7 $\times$) because the file is 2 $\times$ larger than Parquet. This is due to the same reasons described in Sect. 6.1.1. Across all formats and for all expressions, we found that the majority of the time (i.e., >90%) is spent on data loading and decoding, whereas the contribution of the execution of the filter condition is minimal.

6.2.2 String predicates

We next evaluate a predicate on a string column in the `customer_demographic` table with 14% selectivity:

`CD_EDUCATION_STATUS = n` (string column filter)

The results are shown in Fig. 27. Parquet is faster than ORC while Arrow with plain string encoding (“Arrow Feather”) is slower in the uncompressed case, but faster than both when compression is enabled. This is because the `customer_demographic` table is small, implying that I/O is not a bottleneck for this experiment. As a result decompression dominates overall cost and Arrow Feather outperforms because it avoids the cost of decoding. String filtering on Parquet is faster than on ORC. In fact, ORC's bulk loading data access interface requires more string copying than Parquet because it materializes all strings into memory before filtering. Conversely, Parquet's streaming-style data access API does not require keeping all the strings in memory while filtering. Enabling Arrow DICT encoding (“Arrow Feather (DICT)”) marginally improves performance because Arrow decodes everything when loading.²

6.2.3 Bit-vector evaluation

The previous sections produced a bit vector mask that indicates which entries match a given predicate. In this section we look into the performance of applying these bitmaps to produce a result. We start with Zstd-compressed data on disk and, for each format, load a column $C = \langle c_1, \dots, c_n \rangle$ into an in-memory representation. We then mask C using a randomly-generated bit vector $B = \langle b_1, \dots, b_n \rangle$ to produce a result $R = \langle c_i \mid b_i = 1 \rangle$.

² We discuss the “Arrow Feather (Direct)” bar in Sect. 8.1.1.

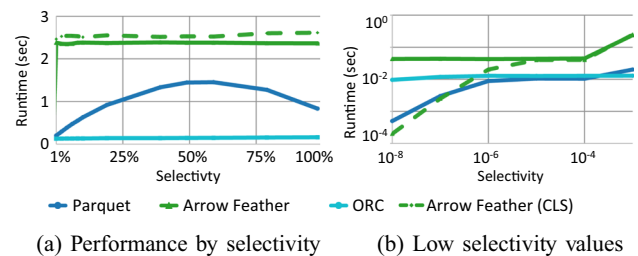


Fig. 28 Bit-vector application performance by selectivity

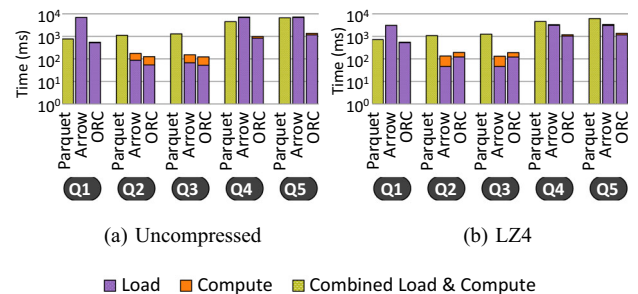


Fig. 29 Log-scale runtimes by format for Table 7 queries with a cold cache



Fig. 30 Table 7 runtimes for cold and warm caches on LZ4 compressed data (Parquet not shown; changes were negligible)

Figure 28a shows performance for each format at various selectivity levels (i.e., at selectivity s , $\sum b_i = s \cdot n$) on the `CS_SOLD_TIME_SK` integer column of the `catalog_sales` table. We can see Arrow Feather and ORC runtimes are approximately constant across all selectivity levels (though ORC is far faster). This is because both formats load all data into their in-memory structure before extracting the target records. Conversely, instead of fully loading all data, Parquet “pushes down” the operation by decoding only the target records that pass the filter condition. Because of this, Parquet runtimes vary for different selectivity levels. However, Parquet runtime is not a simple linear function of selectivity level. Instead, we observe the highest runtime for this format at ~ 0.5 selectivity, the point at which the largest number of branch mispredictions occur.

Overall, at all but the lowest selectivities ORC performs best. However, if we “zoom into” Fig. 28a at the very lowest selectivities (i.e., we approach point selection) a differ-

Table 7 Evaluated TPC-DS SP query subexpressions

Q1	SELECT cs_ship_date_sk, cs_bill_customer_sk FROM catalog_sales WHERE cs_sold_time_sk=12032 AND cs_sold_date_sk=2452653
Q2	SELECT cd_demo_sk, cd_dep_college_count FROM customer_demographics WHERE cd_gender = ' F' AND cd_education_status = ' Secondary'
Q3	SELECT cd_demo_sk FROM customer_demographics WHERE cd_gender = ' M' AND cd_marital_status = ' D' AND cd_education_status = ' College'
Q4	SELECT cs_ext_sales_price, cs_sold_date_sk, cs_item_sk FROM catalog_sales WHERE cs_wholesale_cost > 80.0 AND cs_ext_tax < 500.0
Q5	SELECT cs_ext_sales_price, cs_sold_date_sk, cs_item_sk, cs_net_paid_inc_tax, cs_net_paid_inc_ship_tax, cs_net_profit FROM catalog_sales WHERE cs_wholesale_cost > 80

ent pattern emerges. As shown in Fig. 28b, at extremely low selectivity levels (i.e., ≤ 0.001), Parquet performs better because it supports fine-grained record level data skipping. Conversely, ORC becomes better than Parquet at slightly higher selectivity (~ 0.01) since ORC provides a dedicated in-memory representation that efficiently loads batches. Specifically, ORC data loading consumes full data blocks, incurring extra overhead for queries with low selectivity where few entries are evaluated. On the other hand, this cost is amortized for queries with high selectivity where more data entries pass the predicate. We also implement an advanced Arrow variant that supports chunk level skipping (CLS), which we discuss in Sect. 8.1.3.

To summarize, trade-offs exist for simple data access operations, with no format being the best in every case. Data skipping is important, but it does not always help. Record-level data access APIs provide flexibility for data skipping but has reduced performance on queries with high selectivity compared to bulk loading APIs. To improve performance, on-disk formats should be more adaptive and co-designed with an in-memory representation.

7 Leaf subexpression evaluation

In this section we bring together the previous microbenchmarks and explore how select/project (SP) subexpressions found at the leaves of a query plan can be directly evaluated on each storage format. We use the standard API provided by each format. For Parquet, we use its streaming-style API to parse, decompress and decode the data entries while, interleaved, we evaluate the queries. For Arrow Feather and ORC,

we load the data into their in-memory representations before applying the query.

We select five representative SP subexpressions from TPC-DS (see Table 7) representing a wide variety of use cases. They project few (Q1, Q3) and many (Q5) columns. They contain both equality (Q1–Q3) and range predicates (Q4–Q5). They contain predicates on integers (Q1), strings (Q2–Q3), and doubles (Q4–Q5). Queries have low (Q5), medium (Q2–Q4), and high (Q1) selectivities.

We execute each subexpression: (i) for each format, (ii) with and without compression (LZ4), and (iii) with and without clearing the system cache (to simulate the cases with repeated queries on the hot data, and infrequent queries on cold files, respectively).

The results are shown in Fig. 29 (uncompressed vs LZ4), and Fig. 30 (warm vs cold cache). Since, both Arrow and ORC provide a custom data loading interface (including parsing, decompressing, and decoding the data into their in-memory representations) before any query evaluation (see Sect. 3), we separately report “Load” and “Query” runtimes for each phase. Parquet pipelines data loading and computation, so we report only the total runtime for this format.

Overall, ORC performs best in terms of query performance because of its efficient in-memory mapping representation (lower loading time for large files) and more data-skipping opportunities resulting from a smaller row batch size. With the default setting, ORC and Arrow have 14,064 and 228 batches for the `catalog_sales` table, and 1,876 and 1 batches for the `customer_demographic` table, respectively. ORC’s finer granularity allows it to skip more entries when no qualified item satisfies the filter condition. Conversely, Arrow loading time dominates runtime,

slowing query evaluation, as we also observed in Fig. 24. Parquet outperforms Arrow for Q1, Q4 and Q5 over large tables since Parquet file has a smaller size (i.e., less I/O) when loading the file. Parquet lags when compression is enabled as Arrow Feather sizes (I/O) decrease.

In Fig. 30, we also see that other than Q2 and Q3 (which are both evaluated on the smaller *customer_demographic* table) both Arrow Feather and ORC are impacted by the system cache (significantly so for Arrow). This is because loading data from disk and building the required in-memory data structures is expensive (as we also discussed in Sect. 6.1.1).

In summary, smaller batches allow for more data skipping, but at the cost of space overhead and increased complexity. Workload-aware data partitioning can be used to balance these factors.

8 Advanced optimizations

Given that there has been extensive discussion about pushing the limits of common open formats [20, 57, 58], the goal of this section is to evaluate the amenability of the Arrow and Parquet formats to support more advanced and emerging optimizations of execution engines. We will first discuss optimizations related to Arrow (Sect. 8.1) and then turn into Parquet (Sect. 8.2). While we focus on Arrow and Parquet because of space constraints and their relatively wide adoption, similar optimizations could also be applied to ORC.

8.1 Optimizing arrow

We first evaluate the feasibility and effectiveness of integrating new optimizations into Arrow: namely its ability to support direct querying [18, 25] (Sect. 8.1.1), and data skipping (Sect. 8.1.3). We then compare the performance of this hand-optimized variant against Gandiva [27], a LLVM-based backend for Arrow (Sect. 8.1.2).

8.1.1 Direct filtering over string columns

We begin by exploring Arrow filtering pushdown (i.e., direct querying) into the encoded space for string columns (we separately evaluate a Parquet variant in Sect. 8.2.1). We modify Arrow to implement direct querying as follows. For each data chunk, we decompress and extract the dictionary from the metadata. We then map the string constant in the query predicate from the string domain into the encoded integer domain of the extracted dictionary. This process allows us to: (i) transform string comparisons into integer comparisons, which can be executed efficiently; and (ii) decode only the records admitted by the predicate.

We evaluate by repeating the experiment described in Sect. 6.2.2. The result is shown as the “Arrow Feather

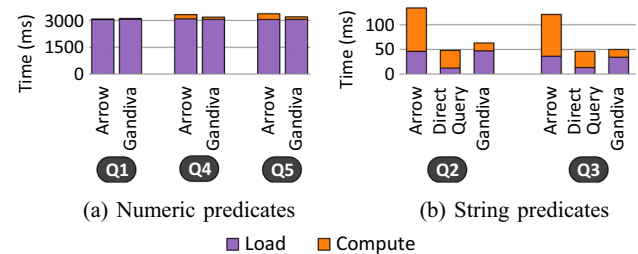


Fig. 31 Arrow Feather runtime with and without direct query (warm cache execution)

(Direct)” bar in Fig. 27. Our results demonstrate a $2\times$ to $4\times$ improvement over other formats. The approach could be extended to range queries by employing an order-preserving dictionary (e.g., as explored in [40]).

8.1.2 Gandiva

Gandiva [27] is an LLVM-based execution backend for Apache Arrow. It is part of the Arrow project, and employs a number of optimizations (e.g., vectorization) applied via LLVM compiler passes. Gandiva further improves performance (especially for string and binary data types) by maximizing zero-copy operations.³

We test Gandiva by executing the queries evaluated in Sect. 7 (listed in Table 7). To do so, we construct expression trees for each query using *TreeExprBuilder* instances, which Gandiva transforms into machine code. As illustrated in Fig. 31, Gandiva is able to slightly reduce computation time for all queries, while loading time (which is the dominant component of the queries with numeric predicates) remains unchanged. For queries with string predicates (i.e., Q2–Q3), we further compare with the optimized Arrow direct query variant described in Sect. 8.1.1. For these queries, the direct query variant achieves $3\times$ speedup compared to vanilla Arrow and outperforms Gandiva as well. This is because direct query not only reduces compute time, but data loading also improves because the data decoding step is skipped.

Interestingly, Gandiva fails to generate vectorized versions for any of the evaluated queries. To test this aspect, we use Gandiva to execute a variant of Q4 that it was able to vectorize:

```
SELECT CS_EXT_LIST_PRICE -
       CS_EXT_WHOLESALE_COST -
       CS_EXT_DISCOUNT_AMT +
       CS_EXT_SALES_PRICE
FROM CATALOG_SALES
```

³ The Parquet and ORC APIs could benefit from reducing or eliminating redundant string duplication.

Gandiva's use of vectorization (and other LLVM optimizations) resulted in a $1.8\times$ speedup (74ms vs 42ms) relative to normal Arrow.

Finally, we observe that Gandiva's compilation process is expensive relative to execution time. For the queries on the smaller dataset (i.e., Q2–Q3), the compilation time exceeded execution time (e.g., for Q2 compilation time was 103ms and runtime was 79ms).

8.1.3 Data skipping

In Sect. 6.2.3 we showed how Parquet achieves excellent performance for low-selectivity filters by avoiding decode overhead for unnecessary records. We implement a similar technique for Arrow. Specifically, we augmented the bulk loading API in Arrow with the data skipping approach leveraged in Parquet. To do so, we modified the Arrow Feather API to support chunk-level skipping (CLS) where we only load the column chunks necessary to answer a given query. Because of its data layout, CLS is the most granular skipping we can employ for Arrow Feather.

To evaluate, we repeat the experiment described in Sect. 6.2.3, which requests a random set of row IDs. We show the result in Fig. 28b as the “Arrow Feather (CLS)” series. As we see in the figure, this variant initially performs well but quickly degrades to perform similarly to unmodified Arrow. This is because the input is a bit-vector with random row IDs: even at extremely low selectivities, we quickly select at least one tuple per chunk, obviating any performance advantages.

8.2 Optimizing parquet

In this section, we build on the lessons learned throughout the paper and augment Parquet with an efficient in-memory representation as well as vectorized instructions (Sect. 8.2.1). Finally, we put everything together and wrap up with an experimental evaluation comparing all optimizations (Sect. 8.2.2).

8.2.1 In-memory parquet and vectorization

In columnar databases, it is common to encode data for better memory and bandwidth utilization. Therefore Parquet (and ORC) could potentially be leveraged as a useful *in-memory* data structure, without transcoding (to Arrow). Parquet's existing data access API either (i) fully deserializes data (precluding opportunities for fine-grained skipping or direct querying) or (ii) exposes record-level data access, which is typically much less efficient than batch loading. OLAP systems could benefit from an access pattern falling in between these two.

One example of this is CodecDB [25], which introduced a dedicated in-memory representation for Parquet.

In CodecDB, Parquet data is lazily materialized (memory mapped) and fully decoded only when needed. This enables support for row batch-level, column chunk-level, and record-level skipping. Lazily-decompressed in-memory Parquet can be further optimized by leveraging vectorized instructions over in-place encoded data, as described in SBoost [23]. We implemented these optimizations based on the code as provided by CodecDB and SBoost. We then compare the scalar performance with direct query, vectorization, and their combinations.

8.2.2 Results

We apply the above optimizations to Parquet and evaluate the performance over the queries listed in Table 7. The results are in Fig. 32. As baselines, we show Parquet with its default streaming API (“Parquet”). We also show a variant in which we load Parquet into Arrow Table before query execution (“P-ArrowTable”); we included this second baseline because in Sect. 7 we observed that loading data into an in-memory data structure can be quite efficient. Next, “P-IM” is the Parquet variant augmented with the in-memory format described in Sect. 8.2.1. “P-IM+D” further adds direct querying in the encoded domain. Finally, “P-IM+D+SIMD” enables SIMD instructions directly in the encoded domain.

As we can see from the figure, P-ArrowTable outperforms the streaming API. Nevertheless, Arrow still requires fully decoding the data into its table format, which limits the optimization opportunities. In fact, P-IM shows even better performance than P-ArrowTable, and achieves more than one order of magnitude improvement over the Parquet baseline because of its lazy materialization avoiding unnecessary decoding overhead from Parquet into Arrow. This can be further improved with direct query and avoiding decoding; P-IM+D is up to $60\times$ faster than the Parquet baseline. Finally, if we enable SIMD execution (AVX_512) we observe up to $100\times$ speedup compared with the Parquet baseline.

Overall, there is huge potential for query speedup when we push the query operator further down into the on-disk format and encoded domain, which demonstrates the feasibility of query push-down in the storage format when augmented with a corresponding optimized in-memory representation.

9 Related work

9.1 Columnar data format trade-offs

There has been substantial recent discussion both online (e.g., [9, 10] and in the research community [8, 59] about the benefits and drawbacks of having multiple, often-overlapping open formats for representing columnar data. Differently than [59] our goal is not to propose a new format. Differently

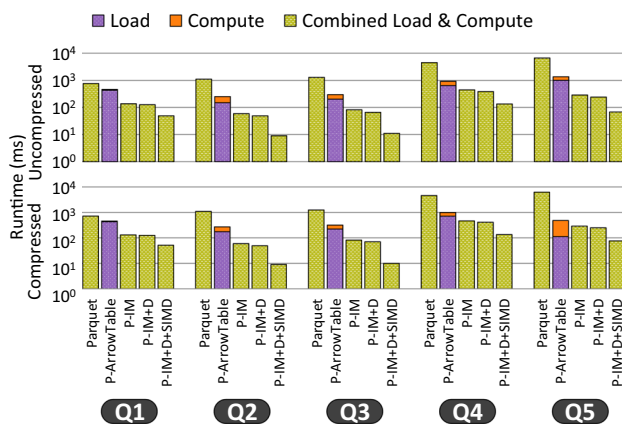


Fig. 32 Parquet performance with various optimizations

than [8] and other SQL-over-Hadoop evaluations [60, 61] our goal is not to evaluate the end-to-end performance of big data systems, but rather to understand how these format can be leveraged as native formats in analytical DBMSs (and how close they conform to columnar RDBMSs standards) [13, 18]. Our takeaways are consistent with the observations of Abadi [9]. Nevertheless, we provide an updated view of the trade-offs in these formats. Interestingly, we find that several limitations described in [9] persist to this day, despite their being highlighted more than six years ago.

9.2 Other encodings

In addition to the encoding methods discussed in Sect. 2.2, *delta encoding* is a basic encoding supported by many columnar data formats. Delta encoding works on integer data. Because differences between adjacent numbers are generally smaller than the numbers themselves, delta encoding greatly reduces data redundancy. Delta encoding works best when the numbers are large, but the value range is small. Direct querying on delta encoded data is challenging because sequential decoding is required to recover a specific record. Delta encoding variations, such as FOR and PFOR [62] use a fixed reference value instead of the previous value, which better serves the direct query on the encoded format. Even though Parquet and ORC support these delta-like encodings, the formats never elected to employ them in our experiments, presumably due to reduced performance or sub-optimal encoding selection.

9.3 Text formats

In addition to the aforementioned binary file formats, text-encoded formats such as CSV and JSON remain highly-utilized. Such formats offer the advantage of being human-readable, but are far more limited in terms of compression. A comma-separated values (CSV) [63] file is a delimited

text file where each line represents a data record, CSV is a common data exchange format supported by many data applications and systems. Conversely, JSON [64] interleaves schema and data. JSON supports nestable and complex data structures. This interleaving better-supports nesting and complex data layouts, whereas it is less space-efficient relative to CSV because of this schema overhead.

9.4 Binary formats

Apache CarbonData [65] is an indexed columnar data format for analytics. Similar to Parquet, it uses compression and encoding to improve efficiency. Apache Avro [66] is a row-oriented storage format. It stores schema as JSON in the file header, making it an excellent choice for schema evolution tasks and write-heavy data operations such as whole-row consumption and processing. Avro also supports serialization and block level compression. We did not evaluate these formats in the paper since they are either not columnar or rarely employed by OLAP systems.

9.5 Domain-specific binary formats

Serialization formats such as Pickle, PyTorch's PT, and NumPy's NPY, are also widely used in domain-specific contexts. These formats are particularly valuable in Python-based applications and machine learning workflows. Pickle is a Python-centric serialization library used to transform Python object instances into a byte stream (and vice versa), which is useful for saving and loading data structures or models directly in Python [67]. PyTorch's PT format is used to save trained models and their parameters, typically as binary files that contain a model's architecture and weights [68]. NumPy's NPY format is employed to save arrays of data, storing the array's data, shape, and datatype efficiently [69]. Bullion [70] is a specialized columnar storage system engineered to address the unique demands of machine learning workloads. Vortex [71] is a columnar file format proposed as a successor to Apache Parquet, offering significantly faster random reads and scans without compromising compression efficiency. Vortex format is extensible and borrows ideas from BtrBlocks [36] and FastLanes [72]. Nimble [73], currently closely coupled with Velox, is a high-speed data processing library intended for future decoupling to expand its flexibility. Lance is a recent format introducing specialized optimizations for handling large-blob columns in machine learning workflows, enhancing both random access and scan efficiencies in such specialized data applications [55]. Lance is built on top of Apache Arrow. Finally, Hierarchical Data Format 5 (HDF5) is a flexible format capable of handling a wide range of storage needs and is extensively used across various scientific disciplines due to its robust handling of large and multidimensional datasets [74].

While there are numerous columnar data formats proposed in the literature, this work deliberately focuses on Arrow, Parquet, and ORC because they are de facto standards with broad community adoption, robust production-grade optimizations, and real-world trade-offs that support general-purpose OLAP scenarios. Moreover, their well-tested cross-system interoperability positions them as especially relevant to both practitioners and researchers. Other formats may demonstrate distinct advantages under specialized scenarios; however, a thorough exploration of all these alternatives would far exceed the scope of this study. By concentrating our evaluation on Arrow, Parquet, and ORC, we strike a balance between broad applicability and in-depth analysis.

9.6 Other optimizations

The latest version of Parquet introduces the concept of an index page that contains statistics for data pages and can be used to skip pages when scanning data in ordered and unordered columns [75]. Even with the newly added index page to facilitate the access of data entries, Parquet's random-access operations are still costly in general compared with Arrow. Velox [56] is a recent effort trying to unify the execution layer across analytical engines. As also highlighted in this paper, Velox recognizes that Apache Arrow limited support for encodings is not a good fit for performant analytical engines. Velox therefore proposes improvements on top of Arrow for addressing this gap. It also highlights the need for a smart partition policy where data layout is organized for a given task (e.g., so that many data blocks can be skipped).

Other recent workload-driven data partition approaches such as Qd-tree [76], SDCs [77], Jigsaw [78] and Pixels [79] address the partition problem by focusing on data access patterns. Differently, in this paper we focus on workload-agnostic columnar formats without considering any workload-specific techniques. Workload-driven data partitioning is an exciting area (further motivated by our experimental results) and could be a promising direction for future work. The co-design of query engine and columnar formats should take workload-driven partitioning into consideration to enhance query performance (e.g., through efficient data skipping and improved compression).

10 Conclusion

In this paper, we evaluated three widely-used open columnar formats. Our systematic assessment employed micro-benchmarks that included evaluating compression performance across real-world scalar and vector datasets, basic database operations, and end-to-end query subexpression execution. We found trade-offs that make each format more or less suitable for use as an internal format in a DBMS. By

applying various optimizations, we identified opportunities to more holistically co-design a unified in-memory and on-disk data representation for better query execution in modern OLAP systems.

Funding Open Access funding provided by the MIT Libraries.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Ramakrishnan, R., Sridharan, B., Douceur, J.R., Kasturi, P., Krishnamachari-SamPATH, B., Krishnamoorthy, K., Li, P., Manu, M., Michaylov, S., Ramos, R., Sharman, N., Xu, Z., Barakat, Y., Douglas, C., Draves, R., Naidu, S.S., Shastry, S., Sikaria, A., Sun, S., Venkatesan, R.: Azure data lake store: A hyperscale distributed file service for big data analytics. In SIGMOD, 2017, pages 51–63
2. Apache Software Foundation. Apache parquet. parquet.apache.org. Accessed: 2024
3. Apache Software Foundation. Apache ORC. orc.apache.org. Accessed: 2024
4. Armbrust, M., Ghodsi, A., Xin, R., Zaharia, M.: Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics. In CIDR, 2021
5. Apache Software Foundation. Apache arrow. arrow.apache.org. Accessed: 2024
6. Rodriguez, S.A., Chackrabroty, J., Chu, A., Jimenez, I., LeFevre, J., Maltzahn, C., Uta, A.: Zero-cost, Arrow-enabled data interface for Apache Spark. In Big Data, 2021, pages 2400–2405
7. Dremio. dremio.com. Accessed: 2024
8. Ivanov, T., Pergolesi, M.: The impact of columnar file formats on SQL-on-Hadoop engine performance: A study on ORC and parquet. CCPE 32(5), e5523 (2020)
9. Abadi, D.: Apache arrow versus parquet and ORC: do we really need a third apache project for columnar data representation? dbmsmusings.blogspot.com/2017/10/apache-arrow-vs-parquet-and-orc-do-we.html. Accessed: 2024
10. McKinney, W.: Some comments to Daniel Abadi's blog about Apache Arrow. wesmckinney.com/blog/arrow-columnar-abadi, (Nov 2017). Accessed: 2024
11. Zeng, X., Hui, Y., Shen, J., Pavlo, A., McKinney, W., Zhang, H.: An empirical evaluation of columnar storage formats. Proc. VLDB Endow. 17(2), 148–161 (2023)
12. Li, T., Butrovich, M., Ngom, A., Lim, W.S., McKinney, W., Pavlo, A.: Mainlining databases: supporting fast transactional workloads on universal columnar data file formats. VLDB 14(4), 534–546 (2020)
13. Ferrari, A., Russo, M.: The definitive guide to DAX: Business intelligence with Microsoft Excel, SQL server analysis services, and Power BI. Microsoft Press, 2015

14. Abadi, D., Boncz, P.A., Harizopoulos, S., Idreos, S., Madden, S.: The design and implementation of modern column-oriented database systems. *Found. Trends Databases* **5**(3), 197–280 (2013)
15. Jain, P., Kraft, P., Power, C., Das, T., Stoica, I., Zaharia, M.: Analyzing and comparing lakehouse storage systems. In *CIDR*, 2023
16. Gracia-Tinedo, R., Sanchez-Artigas, M., Garcia-Lopez, P., Moatti, Y., Gluszk, F.: Lambda-flow: automatic pushdown of dataflow operators close to the data. *CCGRID*, 2019, pages 112–121
17. Yang, Y., Youill, M., Woicik, M., Liu, Y., Yu, X., Serafini, M., Aboulmaga, A., Stonebraker, M.: FlexPushdownDB: hybrid pushdown and caching in a cloud DBMS. *VLDB* **14**(11), 2101–2113 (2021)
18. D. J. Abadi, S. R. Madden, and N. Hachem. Column-stores versus row-stores: how different are they really? In *SIGMOD*, 2008, pages 967–980
19. Abadi, D., Madden, S., Ferreira, M.: Integrating compression and execution in column-oriented database systems. *SIGMOD*, 2006, pages 671–682
20. Zhang, F., Wan, W., Zhang, C., Zhai, J., Chai, Y., Li, H., Du, X.: CompressDB: Enabling efficient compressed data direct processing for various databases. In *SIGMOD*, 2022, pages 1655–1669
21. Li, Y., Patel, J.M.: BitWeaving: fast scans for main memory data processing. In *SIGMOD*, 2013, pages 289–300
22. Hentschel, B., Kester, M.S., Idreos, S.: Column sketches: A scan accelerator for rapid and robust predicate evaluation. In *SIGMOD*, 2018, pages 857–872
23. Jiang, H., Elmore, A.J.: Boosting data filtering on columnar encoding with SIMD. In *DaMoN*, 2018, pages 1–10
24. Wang, Z., Kara, K., Zhang, H., Alonso, G., Mutlu, O., Zhang, C.: Accelerating generalized linear models with MLWeaving: a one-size-fits-all system for any-precision learning. *VLDB* **12**(7), 807–821 (2019)
25. Jiang, H., Liu, C., Paparrizos, J., Chien, A.A., Ma, J., Elmore, A.J.: Good to the last bit: data-driven encoding with CodecDB. In *SIGMOD*, 2021, pages 843–856
26. Liu, C., Jiang, H., Paparrizos, J., Elmore, A.J.: Decomposed bounded floats for fast compression and queries. *VLDB* **14**(11), 2586–2598 (2021)
27. Pindikura, R.: Gandiva. dremio.com/blog/announcing-gandiva-initiative-for-apache-arrow. Accessed: 2024
28. Graefe, G., Shapiro, L.D.: Data compression and database performance. University of Colorado, Boulder, Department of Computer Science, 1990
29. Roth, M.A., Van Horn, S.J.: Database compression. *SIGMOD* **22**(3), 31–39 (1993)
30. Paparrizos, J., Liu, C., Barbarioli, B., Hwang, J., Edian, I., Elmore, A.J., Franklin, M.J., Krishnan, S.: VergeDB: A database for IoT analytics on edge devices. In *CIDR*, 2021
31. Deutsch P. et al.: Gzip file format specification version 4.3. Technical report, RFC 1952, 1996
32. Google. Snappy: a fast compressor/decompressor. google.github.io/snappy. Accessed: 2024
33. Gailly, J.-L., Adler, M.: Zlib compression library. 2004
34. Shi, J.: Column partition and permutation for run length encoding in columnar databases. In: *SIGMOD*, 2020, pages 2873–2874
35. Liu, C.: Fast and effective compression for iot systems, 2022
36. Kuschewski, M., Sauerwein, D., Alhomssi, A., Leis, V.: Btrblocks: efficient columnar compression for data lakes. *Proceed. ACM Manag. Data* **1**(2), 1–26 (2023)
37. Lindstrom, P., Isenbarg, M.: Fast and efficient compression of floating-point data. *IEEE Trans. Visual Comput. Graphics* **12**(5), 1245–1250 (2006)
38. Gupta, A., Agarwal, D., Tan, D., Kulesza, J., Pathak, R., Stefani, S., Srinivasan, V.: Amazon Redshift and the case for simpler data warehouses. In *SIGMOD*, 2015, pages 1917–1923
39. Larson, P.-Å., Clinciu, C., Hanson, E.N., Oks, A., Price, S.L., Rangarajan, S., Surna, A., Zhou, Q.: SQL Server column store indexes. In *SIGMOD*, 2011, pages 1177–1184
40. Liu, C., Umbenhowe, M., Jiang, H., Subramaniam, P., Ma, J., Elmore, A.J.: Mostly order preserving dictionaries. In *ICDE*, 2019, pages 1214–1225
41. Jiang, H., Liu, C., Jin, Q., Paparrizos, J., Elmore, A.J.: PIDS: attribute decomposition for improved compression and query performance in columnar storage. *VLDB* **13**(6), 925–938 (2020)
42. Liu, C., Paparrizos, J., Elmore, A.J.: Adaedge: A dynamic compression selection framework for resource constrained devices. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pages 1506–1519
43. Boncz, P., Neumann, T., Leis, V.: FSST: fast random access string compression. *VLDB* **13**, 2649–2661 (2020)
44. Zeng, X., Meng, R., Pavlo, A., McKinney, W., Zhang, H.: Nulls!: Revisiting null representation in modern columnar formats. In *Proceedings of the 20th international workshop on data management on new hardware*, 2024, pages 1–10
45. Apache Software Foundation. Apache feather. arrow.apache.org/docs/python/feather.html. Accessed: 2024
46. Melnik, S., Gubarev, A., Long, J.J., Romer, G., Shivakumar, S., Tolton, M., Vassilakis, T.: Dremel: interactive analysis of web-scale datasets. *VLDB* **3**(1–2), 330–339 (2010)
47. What is the difference between apache drill's valuevectors and apache arrow? - stack overflow. stackoverflow.com/questions/53533506/what-is-the-difference-between-apache-drills-aluevectors-and-apache-arrow. (Accessed on 09/18/2024)
48. Join Order Benchmark (JOB). github.com/greghahn/join-order-benchmark. [Accessed: 2024]
49. Public BI benchmark. github.com/cwida/public_bi_benchmark. [Accessed: 2024]
50. Datasets. huggingface.co/docs/datasets/index. (Accessed on 09/11/2024)
51. Nambiar, R.O., Poess, M.: The making of TPC-DS. In *VLDB*, 2006, pages 1049–1058
52. Li, X., Li, J.: Angle-optimized text embeddings, 2023. arXiv preprint [arXiv:2309.12871](https://arxiv.org/abs/2309.12871)
53. Rui Meng, S.R.J.C.X.Y.Z.S.Y., Liu, Ye: Sfr-embedding-mistral: enhance text retrieval with transfer learning. *Salesforce AI Research Blog*, 2024
54. Pyarrow - apache arrow python bindings v17.0.0. arrow.apache.org/docs/python/index.html. (Accessed on 09/20/2024)
55. lance. github.com/lancedb/lance. (Accessed on 09/19/2024)
56. Pedreira, P., Erling, O., Basmanova, M., Wilfong, K., Sakka, L.S., Pai, K., He, W., Chattopadhyay, B.: Velox: meta's unified execution engine. *VLDB* **15**(12), 3372–3384 (2022)
57. InfluxData. Querying Parquet with millisecond latency. influx-data.com/blog/querying-parquet-millisecond-latency, (December 2022). Accessed: 2024
58. Agarwal, R., Khandelwal, A., Stoica, I.: Succinct: Enabling queries on compressed data. In *NSDI*, 2015, pages 337–350
59. Trivedi, A., Stuedi, P., Pfefferle, J., Schuepbach, A., Metzler, B.: Albis: High-performance file format for big data systems. In *USENIX*, 2018, page 615–629
60. Floratos, A., Minhas, U.F., Özcan, F.: SQL-on-Hadoop: Full circle back to shared-nothing database architectures. *VLDB* **7**(12), 1295–1306 (2014)
61. Pirzadeh, P., Carey, M., Westmann, T.: A performance study of big data analytics platforms. In *Big Data*, 2017, pages 2911–2920
62. Lemire, D., Boytsov, L.: Decoding billions of integers per second through vectorization. *Softw. Pract. Exp.* **45**(1), 1–29 (2015)
63. Comma-separated values. en.wikipedia.org/wiki/Comma-separated_values. Accessed: 2024

64. Json (javascript object notation). www.json.org/json-en.html. Accessed: 2024
65. Apache Software Foundation. Apache carbonData. carbon-data.apache.org. Accessed: 2024
66. Vohra, D.: Apache Avro. In *Practical Hadoop Ecosystem*, 2016, pages 303–323
67. pickle - python object serialization - python 3.12.6 documentation. docs.python.org/3/library/pickle.html. (Accessed on 09/18/2024)
68. Saving and loading models for inference in pytorch. pytorch.org/tutorials/recipes/recipes/saving_and_loading_models_for_inference.html. (Accessed on 09/18/2024)
69. Npy format. numpy.org/devdocs/reference/generated/numpy.libformat.html. (Accessed on 09/18/2024)
70. Liao, G., Liu, Y., Chen, J., Abadi, D.J.: Bullion: A column store for machine learning. In *Proceedings of the 15th Annual Conference on Innovative Data Systems Research (CIDR '25)*, Amsterdam, The Netherlands, Jan 19–22, 2025
71. Vortex. github.com/spiraldb/vortex. (Accessed on 02/19/2025)
72. Afroozeh, A., Boncz, P.: The fastlanes compression layout: Decoding > 100 billion integers per second with scalar code. *Proceed. VLDB Endow.* **16**(9), 2132–2144 (2023)
73. The nimble file format. github.com/facebookincubator/nimble. (Accessed on 02/19/2025)
74. The hdf5® library and file format. www.hdfgroup.org/solutions/hdf5/. (Accessed on 09/20/2024)
75. Apache Software Foundation. ColumnIndex layout to support page skipping. github.com/apache/parquet-format/blob/master/PageIndex.md. Accessed: 2024
76. Yang, Z., Chandramouli, B., Wang, C., Gehrke, J., Li, Y., Minhas, U.F., Larson, P.-Å., Kossmann, D., Acharya, R.: Qd-tree: Learning data layouts for big data analytics. In *SIGMOD*, 2020, pages 193–208
77. Madden, S., Ding, J., Kraska, T., Sudhir, S., Cohen, D., Mattson, T., Tatbul, N.: Self-organizing data containers. *Memory* **1**, 2 (2022)
78. Kang, D., Jiang, R., Blanas, S.: Jigsaw: A data storage and query processing engine for irregular table partitioning. In *SIGMOD*, 2021, pages 898–911
79. Bian, H., Ailamaki, A.: Pixels: An efficient column store for cloud data lakes. In *ICDE*, 2022, pages 3078–3090

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.